LUMAWAVE

MUSIC DISCOVERY & PLAYER APPLICATION

PROJECT OVERVIEW, TECHNICAL ARCHITECTURE,
FEATURE SPECIFICATION AND IMPLEMENTATION REPORT

Name	Abdullah Azeem
Date	13-10-2026
Title	Lumawave Music App
Registration Number	SEP26-FD25-23

Premium 3D-inspired UI • Supabase • PostgreSQL • Music APIs

Prepared for: Abdullah Azeem

Report date: 13 October 2026

Editable Word document | Times New Roman | Body 16 pt | Headings 18 pt

Document Information and Scope

Item	Details
Project name	LumaWave Music App
Project category	Flutter cross-platform music discovery and playback application
Primary backend recommendation	Supabase Auth, PostgreSQL, Row Level Security, Realtime and Edge Functions
Client technology	Flutter and Dart
Report basis	Prior project requirements and implementation plans in this conversation, plus 12 user-supplied mobile screenshots.
Evidence limitation	The source-code repository, migration files, Supabase project configuration and provider credentials were not included in the uploaded materials. Backend operation and playback cannot therefore be independently certified by this report.
Document format	Editable Microsoft Word (.docx)
Typography	Times New Roman; 16 pt body copy; 18 pt headings; smaller table text used where required for readable tabular layouts.

Important Project Status Note

This report documents the product design, visible interface states and agreed implementation plan. The supplied screenshots visibly show a premium splash screen, sign-in UI, music catalogue/list, Home, Search, Discover, Library, playlist views and profile/theme settings. Some screens also display labels such as “All Songs (278)” and source names. These are treated as screenshot-visible UI claims, not as independent proof that a live API currently returns that count, that every listed track is playable, or that Supabase authentication and database security have already been verified.

Where this report describes database tables, authentication behavior, streaming, lyrics, offline synchronization or automated tests, these are the intended implementation design and acceptance requirements unless explicitly marked as visible in the supplied screenshots.

Contents

- Executive Summary
- 1. Project Overview and Goals
- 2. Scope and User Journeys
- 3. Existing UI Evidence from Supplied Screenshots

- 4. Visual Design System and 3D-Inspired Experience
- 5. Recommended Technology Stack
- 6. System Architecture
- 7. Supabase Database Design
- 8. Authentication and Account Security
- 9. Music Catalogue and Provider Integration
- 10. Audio Playback and Player Behavior
- 11. Lyrics, Search and Discovery
- 12. Offline Support and Synchronization
- 13. Theme, Animation and Responsive Design
- 14. Privacy, Security and Reliability
- 15. Testing Strategy and Acceptance Criteria
- 16. Implementation Roadmap
- 17. Risks, Constraints and Mitigations
- 18. Future Enhancements
- Conclusion
- References and Technical Resources
- Appendix A. Supplied UI Screenshots (all 12 images)

Executive Summary

LumaWave is a premium music discovery and playback application built around a distinctive glossy, 3D-inspired visual identity. Its design direction combines a luminous splash screen, dark immersive surfaces, vibrant cyan and violet accents, rounded glass-like cards, expressive album artwork, responsive navigation and smooth interactions. The supplied screenshots establish a coherent visual direction across account entry, music discovery, search, library, playlists and theme preferences.

The recommended technical foundation is Flutter for the client and Supabase for authentication and persistent application data. Supabase is selected because PostgreSQL is a natural fit for related entities such as profiles, preferences, favorites, playlists, ordered playlist tracks and listening history. Row Level Security can enforce user ownership inside the database, while Edge Functions can protect integrations that require server-side credentials. The official Supabase Flutter quickstart outlines client initialization and recommends enabling RLS and granting only needed privileges. [1][2]

The audio catalogue is designed to combine supported music providers through independent adapters. The app must distinguish authorized full streams, preview-only media, official embedded playback and external handoff. A database stores application data and permitted metadata; it does not grant rights to commercial recordings or lyrics. A target of 200 or more tracks should be measured from actual, unique provider responses, not met by fabricated records.

Offline functionality requires a local cache and deliberate synchronization strategy. Unlike Firestore’s built-in mobile offline document caching, a Supabase-backed app should implement local persistence, queued writes, retries and conflict handling explicitly. [3] The final release gate should include security-rule testing, authentication tests, real provider requests, playback verification, responsive UI checks and performance profiling.

Key Recommendation

Use Supabase as the single primary backend, establish the database schema and RLS policies early, validate the actual music sources, then connect the premium UI to those working foundations. The seven-second splash

animation is a brand requirement; initialize the app and resolve authentication concurrently so the delay does not become an unnecessary network wait.

1. Project Overview and Goals

1.1 Product vision

LumaWave aims to provide an immersive music browsing experience that feels modern and polished while remaining usable, secure and responsive. The application is intended to let users enter through a refined authentication flow, discover music from configured sources, play compatible media, save favorites, organize playlists, view available lyrics and personalize the visual appearance.

1.2 Project objectives

- Create a recognizable original brand, app icon and seven-second animated splash screen.
- Build secure sign-up, sign-in, password recovery, email verification and session restoration using Supabase Auth.
- Persist profiles, themes, favorites, playlists, playlist ordering and listening history in PostgreSQL with ownership controls.
- Integrate real catalogue providers through source-specific adapters and normalize their results into a common track model.
- Provide accurate playback controls that reflect the audio engine state instead of simulated progress.
- Support light, dark and distinctive accent themes across every screen and dialog.
- Create clear online, offline, loading, empty, error and synchronization states.
- Meet the Core Flutter UI Recreation requirement by documenting the role of core layout widgets with comments in the code.
- Test correctness, access control, responsiveness, accessibility and performance before release.

1.3 Target users

The intended users are mobile listeners who want a visually rich music discovery interface, organized personal collections and a simple way to

access music supported by the application's configured sources. The initial priority is Android, with cross-platform Flutter structure maintained so other supported targets can be added deliberately.

1.4 Success criteria

Goal	Success evidence
Authentication	Real registration, verification, sign-in, password recovery and sign-out tested with Supabase.
Data integrity	User-owned rows are protected by RLS; favorites and playlists persist after restart.
Catalogue	Provider results are live, deduplicated and paginated; counts reflect actual records.
Playback	Controls operate a real supported media source; unsupported sources explain their limitation.
User experience	Main screens work across themes and common screen sizes without overflow.
Reliability	Errors are recoverable; no endless loading state or unhandled critical exception remains.
Documentation	Setup, schema, provider capability, limitations and test results are recorded.

2. Scope and User Journeys

2.1 Primary flow

The expected entry sequence is native launch, LumaWave splash animation, authentication-state resolution, and then either Sign In or Home. New users register and complete required verification; returning users restore their existing session. From Home, the user can search or browse, open a compatible track, use the player, save it or add it to a playlist, and later access the collection through Library.

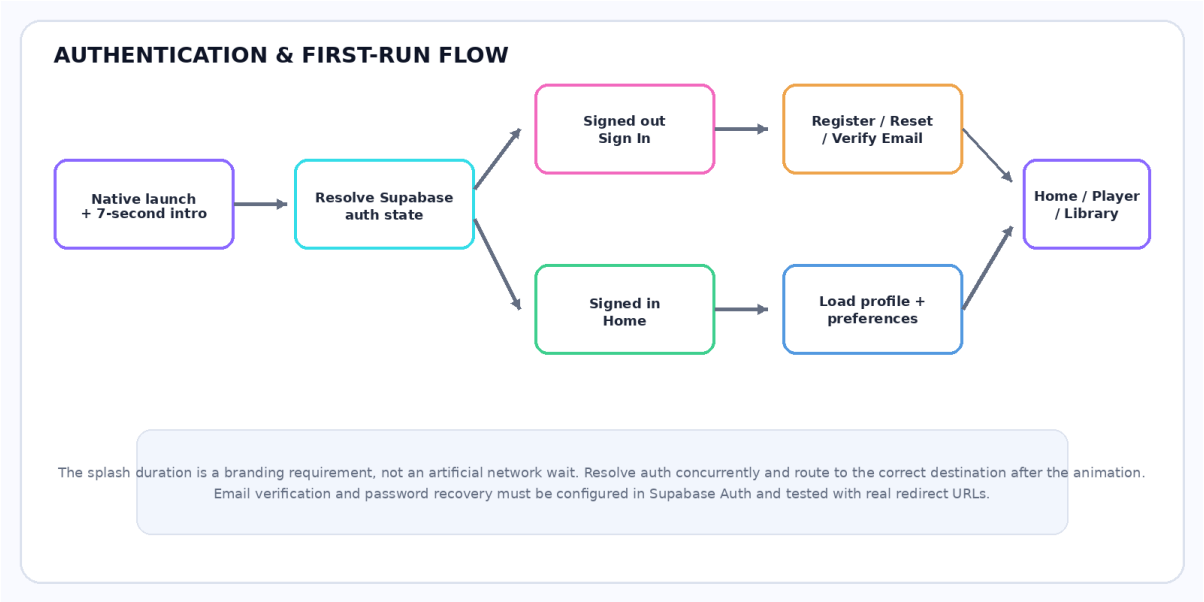


Figure 1. Authentication and first-run flow. Authentication resolution should run concurrently with the brand animation.

2.2 Main application screens

Screen	Main responsibilities	State that must be real
Splash	Branding animation and startup transition.	Seven-second presentation timing and correct auth-based destination.
Sign In	Email/password, Remember me preference, password recovery and registration link.	Supabase Auth result, request loading and clear errors.
Register	Full name, email, password, confirm password and password feedback.	Validated account creation and profile initialization.
Home	Greeting, featured content, track rows, recent activity and mini-player.	Real provider data or a truthful empty state.
Discover	Genres, moods, categories and featured releases.	Categories lead to functional queries or filtered results.
Search	Search input, source filters and results.	Debounced, cancellable and paginated provider requests.
Player	Artwork, track info, seek, queue, repeat, shuffle and lyrics.	Actual audio-engine state and source-appropriate controls.
Library	Favorites, playlists and listening history.	Persistent Supabase data plus local cache where appropriate.
Playlist detail	Cover, Play All, Shuffle, add/remove/reorder.	Owner-checked database operations and eligible playback.
Profile & Themes	Profile details, appearance mode, palette and preferences.	Persisted preferences and consistent theme application.

3. Existing UI Evidence from Supplied Screenshots

Twelve supplied 720 × 1600 pixel screenshots were reviewed and are appended at the end of this report in the order represented by the uploaded archive. They visibly establish a dark, teal-lit design direction with rounded cards and neon accents. This section describes visible UI states only; it does not certify unseen source code, API credentials or backend behavior.

Screenshot evidence	Visible content	Interpretation
Splash	Centered luminous waveform mark, LUMAWAVE wordmark and tagline.	Brand identity and splash-screen direction are established visually.
Sign-in	Welcome message, email and password fields, remember-me, reset link, Sign In and Create Account.	Authentication form presentation is visible; backend correctness requires testing.
Home	Personal greeting, featured artwork, track list, navigation and mini-player area.	Home dashboard layout and music content cards are represented.
Catalogue / track list	Track rows, artwork, metadata and source/capability badges.	Catalogue display has been designed; source labels do not alone prove stream validity.
Discover	Mood and genre cards such as Bollywood, Hollywood and electronic categories.	Genre-led browsing is part of the product concept.
Search	Search field, category pills and track results.	Search screen presentation is visible; query debouncing and pagination remain implementation behaviors to verify.
Library	Liked songs and playlists.	Collection screens are represented; persistence requires database tests.
Playlist details	Playlist identity, track count, Play All and Shuffle actions.	Playlist interaction design is visible; queue behavior must be verified.
Profile & Themes	User card, verified-listener label, appearance-mode options and palette choices.	Theme-control UI is visible; persistence and session verification require tests.
Catalogue count	One screenshot visibly displays “All Songs (278)”.	Treat 278 as displayed UI evidence, not as an independently validated unique live API count.

3.1 Known verification gaps

- The source repository was not included in the upload; code-level widget comments and feature completion cannot be inspected here.
- The Supabase project, SQL migrations and RLS policies were not supplied; database setup and access control cannot be certified from screenshots.

- Music-provider credentials, live response logs and test tracks were not supplied; playback labels must be verified against actual resource URLs and provider terms.
- The screenshots show a polished dark interface; screenshot evidence alone does not prove all light-theme screens, responsive breakpoints, offline synchronization or animation performance.

4. Visual Design System and 3D-Inspired Experience

4.1 Brand language

The design uses a dark, immersive backdrop with cyan/teal light, violet and magenta accents, glossy surfaces and rounded cards. The intended effect is a sense of depth without relying on a full 3D engine. The visual identity should remain original and must not reproduce another streaming service’s logo or imply affiliation.

4.2 Splash sequence

Time window	Animation	Engineering requirement
0.0–0.7 s	Reveal background and ambient glow.	Keep the initial frame light and avoid network-dependent rendering.
0.7–2.0 s	Scale and settle the emblem with a spring-like motion.	Use a small number of optimized layers.
2.0–3.5 s	Reflective sweep and waveform pulse.	Avoid expensive blur stacks; dispose controllers correctly.
3.5–5.5 s	Reveal brand name and tagline.	Maintain readable contrast across device sizes.
5.5–7.0 s	Finish and transition.	Route based on real auth state; do not add another artificial wait.

4.3 Reusable components

- Primary and secondary buttons with enabled, disabled, loading and pressed states.
- Glass-like cards with controlled borders and shadows; avoid using glow as a substitute for hierarchy.
- Artwork tiles with clipping, aspect-ratio constraints, loading placeholders and fallback images.
- Form fields with consistent padding, focus borders, inline validation and accessible labels.

- Persistent mini-player with real playback position and state.
- Genre/mood cards that adapt their grid to available width.
- Dialogs and bottom sheets with consistent transitions, focus handling and safe-area behavior.

4.4 Core Flutter-widget requirement

The assignment specifically requires rebuilding a complex real-world UI using core Flutter layout widgets and explaining each widget in comments. The implementation should use widgets such as SafeArea, Stack, Container, DecoratedBox, Padding, Row, Column, Expanded, Flexible, Wrap, LayoutBuilder, GridView, ListView, CustomScrollView, ClipRRect, Hero, InkWell and native animation primitives. Comments should describe why the widget is selected and which layout or interaction problem it solves; avoid comments that merely restate a class name.

5. Recommended Technology Stack

Layer	Recommended technology	Role / reason
UI client	Flutter + Dart	Cross-platform mobile interface, core-widget recreation and native animation.
Authentication	Supabase Auth	Email/password accounts, sessions, recovery and supported verification/OAuth flows.
Primary database	Supabase PostgreSQL	Relational storage for playlists, ordered track entries, preferences and user activity.
Authorization	PostgreSQL Row Level Security	Enforce user ownership at the database boundary.
Server functions	Supabase Edge Functions	Protected third-party API calls and privileged business operations when required.
Realtime	Supabase Realtime	Selected cross-device changes, e.g. playlist and preference updates.
Local persistence	SQLite via a suitable Flutter library (e.g. Drift)	Metadata cache, queued offline operations and synchronization support.
Audio	Evaluate just_audio / audio_service or a compatible maintained player	Real playback, position events and supported background controls.
Music catalogue	Provider adapters for verified services	Discover real catalogue records while respecting each provider’s capabilities.
Lyrics	Provider adapter, e.g. LRCLIB where available and permitted	Plain or timed lyrics with attribution and unavailable states.
Quality	Flutter tests, integration tests, Supabase SQL/RLS tests	Validate behavior, authorization, UI and release readiness.

5.1 Why Supabase over Firebase for this project?

Supabase is the preferred option because the music app's persistent data is relational: a user owns playlists; each playlist contains ordered track references; favorites and history point to provider identifiers; preferences are tied to a user. PostgreSQL supports foreign keys, uniqueness constraints and expressive joins. Supabase offers an official Flutter quickstart for integrating the client. [1]

Firebase remains a strong alternative, particularly for document-oriented applications that prioritize built-in Firestore offline caching. Firestore automatically manages cached documents and synchronizes local changes when connectivity returns on supported mobile clients. [3] With Supabase, equivalent offline-first user experiences require a designed local database and synchronization policy. For this project, the advantages of SQL relationships and RLS outweigh that additional engineering, provided the offline layer is properly implemented.

5.2 What Supabase does not provide

Supabase does not automatically provide a catalogue of 200+ commercially released songs, legal streaming permissions, full-length audio, or original lyrics. These depend on external providers and their policies. The backend stores user application data and permitted metadata; it is not a replacement for licensed media distribution.

6. System Architecture

The Flutter client should be divided into presentation, application/state, repositories, external services and local persistence. Supabase acts as the single primary backend for identity and durable application records. External music providers remain authoritative for their own catalogue and playback constraints.

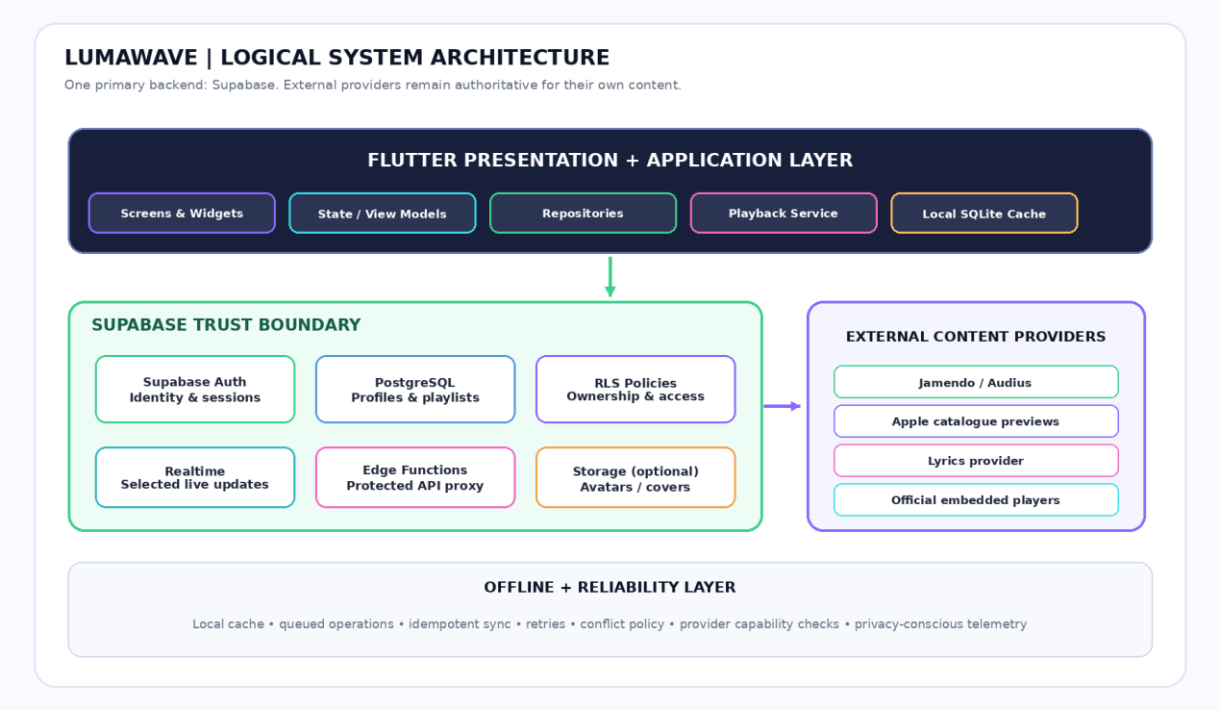


Figure 2. Recommended LumaWave architecture. The client uses publishable configuration and RLS; private provider secrets remain server-side.

6.1 Responsibilities by layer

Layer	Responsibilities	Must not do
Presentation	Render widgets, collect input, show loading/error/empty states and bind state.	Run complex SQL/API orchestration directly inside build methods.
View models / controllers	Expose stable screen state, validation, commands and user-facing errors.	Pretend that a request succeeded before the service confirms success.
Repositories	Define data operations and merge local/remote state where needed.	Hardcode production catalogue results.
Provider adapters	Parse provider responses and map capabilities/errors into common models.	Assume all sources support the same playback method.
Playback service	Own queue, audio engine, position, buffering and completion state.	Advance the progress bar independently of actual playback.
Supabase	Authenticate and persist user-owned application data.	Store private provider secrets in public tables or client code.
Local store	Serve cached permitted records and queue pending writes.	Claim audio is available offline when no licensed media is cached.

6.2 Suggested feature-first folders

```
lib/
  main.dart
  app/ (bootstrap, router, app shell)
  core/ (theme, motion, network, errors, local storage, widgets)
  features/
    auth/ (data, domain, presentation)
    home/
    music/ (models, provider adapters, repositories, player, lyrics)
    search/
    library/
    playlists/
    profile/
    settings/
  generated/ (generated configuration only where appropriate)

supabase/
  migrations/
  tests/ (RLS and database behavior tests)
  functions/ (server-side provider integrations when needed)

test/
  unit/  widget/  integration/
```

7. Supabase Database Design

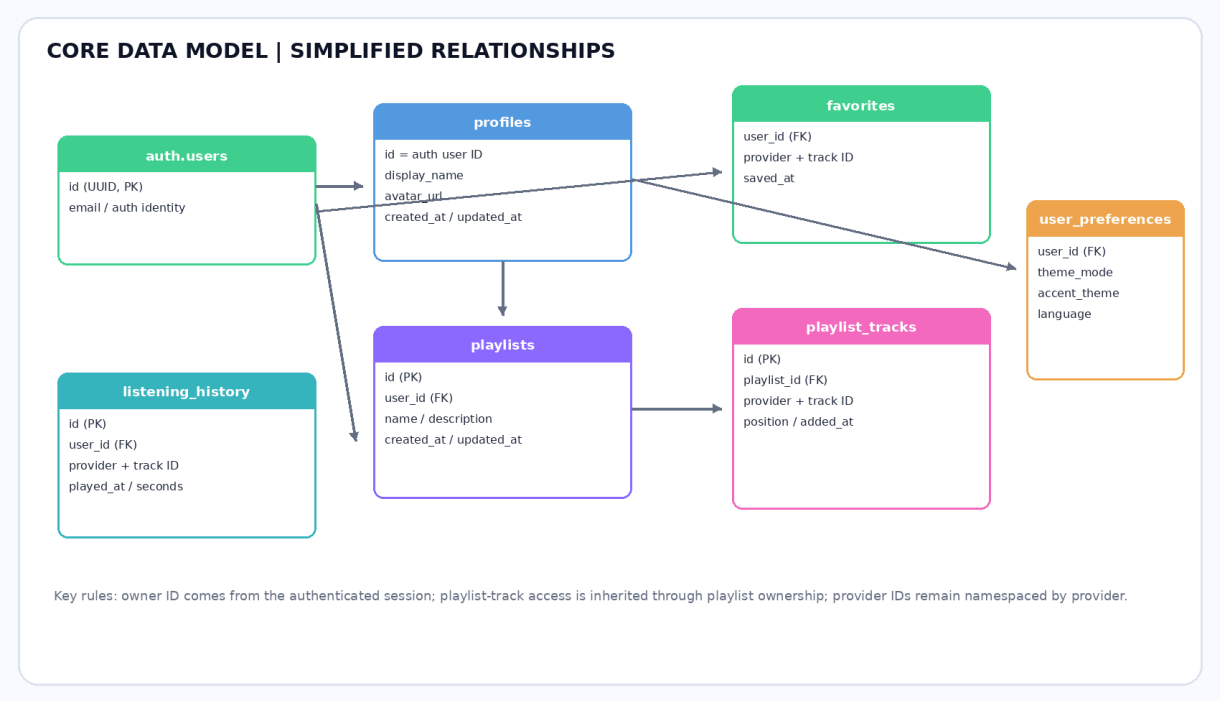


Figure 3. Simplified relationship map for application tables. Provider catalog records are namespaced by provider and provider track ID.

7.1 Proposed schema

Table	Important columns / constraints	Purpose
profiles	id UUID primary key referencing auth user; display_name; avatar_url; timestamps	Public-safe profile information linked to authenticated identity.

Table	Important columns / constraints	Purpose
user_preferences	user_id unique FK; theme_mode; accent_theme; language; reduced_motion; updated_at	User theme and accessibility preferences.
track_catalog_cache	provider; provider_track_id; title; artist; artwork_url; duration_ms; capability metadata; cache timestamps; unique(provider, provider_track_id)	Optional cache of permitted provider metadata.
favorites	user_id FK; provider; provider_track_id; saved_at; unique(user_id, provider, provider_track_id)	Idempotent user favorites.
playlists	id UUID PK; user_id FK; name; description; created_at; updated_at	User-owned named collections.
playlist_tracks	id; playlist_id FK; provider; provider_track_id; position; added_at	Ordered membership for tracks inside playlists.
listening_history	id; user_id FK; provider; provider_track_id; played_at; seconds_played	Recent activity, subject to retention and privacy preferences.
optional user_devices	id; user_id FK; device_label; last_seen_at	Only if device-level session/sync features are required.

7.2 Database design rules

- Use UUIDs for application-owned entities and foreign keys for user/playlist relations.
- Namespace provider identifiers; identical titles do not necessarily identify the same recording.
- Use unique constraints for favorites and playlist entries where appropriate.
- Index common access paths such as user ID, playlist ID, saved time and provider/track keys.
- Use server timestamps for trusted creation/update times where appropriate.
- Validate allowed theme values, field lengths and playlist positions.
- Keep cached provider metadata minimal and within the source provider’s permissions.
- Use migrations in version control rather than editing production schema manually without a reproducible record.

7.3 Row Level Security model

Enable RLS on exposed tables. The authenticated user’s identity must be derived from the Supabase session, not from a client-supplied owner field. Profiles and preferences should be scoped to the owner. Favorites and history must permit only the owner. Playlist-track reads and writes must

check ownership through the parent playlist. Public catalogue reads, if needed, should expose only explicitly allowed metadata. Supabase security guidance states that RLS and least-privilege grants are central to protecting API-accessible data. [2]

Do not embed a secret/service-role key in the Flutter application. Supabase's key guidance explains that secret/service-role keys can bypass RLS and are not safe to expose in client apps. [4] Keep privileged work server-side.

8. Authentication and Account Security

8.1 Sign-in

The sign-in form includes email, password, password visibility, Remember me, Forgot password, a primary action and a registration route. Field-level errors should appear beside the relevant field. The submit button must enter a loading state while Supabase Auth is processing the request and remain disabled against repeat submissions.

8.2 Registration

Registration collects the full name, email, password and confirm-password value. Validate the email, compare password and confirmation values, show live password feedback and apply the configured password policy. Confirmation is never stored. On successful account creation, initialize the profile safely, then follow the project's configured email-verification flow before granting access to features that require a verified account.

8.3 Password recovery and sessions

- "Forgot password?" should send the password-reset flow to the registered email; it does not recover or reveal a password.
- Configure and validate deep links/redirect URLs for email verification and recovery.
- Restore the user's session through Supabase Auth rather than trusting a locally saved "logged in" Boolean.
- Remember me should have a defined, truthful behavior; it must never store the password in plain text.
- Handle expired links, invalid credentials, duplicate accounts, rate limits and unavailable networks clearly.

- On sign-out, clear user-scoped in-memory/local state appropriately so another account cannot see cached private records.
- If account deletion is supported, reauthenticate as needed and plan dependent-data cleanup explicitly.

8.4 Optional hardening

Add provider-based sign-in or multi-factor authentication only after basic email/password flows and redirects are stable. Apply appropriate rate limits, server-side checks, telemetry redaction and secure secret management. Do not claim stronger authentication merely because the UI displays a verification badge.

9. Music Catalogue and Provider Integration

9.1 Provider adapter strategy

Implement one adapter per provider and normalize every response to a common Track entity. The screenshots visibly refer to Jamendo, Audius and iTunes-style sources in the catalogue UI; these names should be matched to the actual configured endpoints and their current access rules in the source code.

Provider candidate	Best-fit role	Important constraint
Audius	Search and stream supported tracks from its catalogue using documented API routes.	Use current developer documentation, current API key/access requirements and track-specific streamability/gating behavior. [5]
Jamendo	Discovery and playback of eligible licensed independent music.	Check the specific track’s license and the use the app intends to make.
Apple iTunes Search	Commercial catalogue metadata, artwork and promotional previews.	Apple treats previews as promotional content with placement, attribution and streaming/no-cache restrictions; it is not a general full-track streaming license. [6]
Lyrics provider	Available plain or timestamped lyrics for matched recordings.	Availability varies; permission to display/cache/reproduce lyrics must be checked.
Optional official player providers	Embedded or external playback for media that cannot be played as a direct stream.	Honor provider API, playback and attribution requirements; do not scrape protected stream URLs.

9.2 Track model

Each track result should include at minimum: provider, provider-specific track ID, title, artist, album where available, artwork URL, duration,

tags/genres, a permitted resource reference, playback capability, attribution information and an optional lyrics capability. Optional properties should remain nullable when providers do not supply them. Resource URLs may expire and must be revalidated.

9.3 200+ dynamic tracks

The product target is a catalogue of at least 200 unique records when the configured sources can supply them. Implement pagination, request cancellation, caching, stable IDs and duplicate detection. The supplied screenshot visibly shows “All Songs (278)”; this is a display captured from the app, not an independently audited API response. A release test must count unique records returned by the live providers and record the result.

- Do not invent artists, tracks or duration values to satisfy a count.
- Do not label a promotional preview as a full stream.
- Do not show a play button that cannot perform a valid provider action.
- Attribute provider data and link to the source when required.
- Distinguish empty search results, rate limits, network failure and unavailable playback.
- Cache only data and artwork to the extent permitted by provider terms.

9.4 Category catalogue

The planned discovery taxonomy includes Bollywood, Hollywood/international, independent music, pop, rock, electronic, classical/instrumental, hip-hop, jazz/world, mood-based collections, new discoveries where available, favorites and listening history. Categories should drive provider queries or filters; they should not just be decorative chips.

10. Audio Playback and Player Behavior

10.1 Playback service

Use one authoritative playback service to own the audio engine, selected track, queue, current index, processing/buffering state, duration, playback position, repeat mode and shuffle state. The expanded player, mini-player and track rows should read from the same state. Evaluate a maintained audio engine compatible with the target platforms; do not make package

selection before confirming current SDK compatibility and required platform setup.

10.2 Required controls

Control	Expected implementation
Play / pause	Invoke actual playback commands and render the engine’s confirmed state.
Progress / seek	Reflect actual position and duration streams; seek only when supported.
Next / previous	Select the next eligible queue item and handle unavailable sources.
Queue	Add, remove and reorder tracks with predictable ordering.
Shuffle / repeat	Implement off, repeat-one and repeat-all; keep shuffle semantics consistent.
Background playback	Configure supported audio-session, notification and media controls per platform.
Completion	Handle end-of-track, queue progression and resource cleanup.
Failures	Show an understandable message for expired, unavailable or unsupported sources and offer a valid fallback.
Mini-player	Show current track, real playback progress and functioning play/pause.
Visualizer	Animate only from real audio data exposed by the engine; do not fake frequency information.

10.3 Performance expectations

Do not rebuild the entire page on every playback-position tick. Isolate the progress/lyrics area, cancel subscriptions when no longer needed and profile artwork transitions and scrolling. Flutter recommends profiling frame performance and minimizing expensive rendering work; profile on representative physical devices rather than relying only on debug mode. [7]

11. Lyrics, Search and Discovery

11.1 Lyrics feature

Lyrics lookup is a separate adapter from audio playback. It should match by title, artist and preferably duration or provider IDs where available. Show synchronized line highlighting only when trustworthy timestamps exist. Otherwise render plain lyrics if available and permitted. Include loading, retry, unavailable and attribution states. LRCLIB publishes an API specification for lyric records, including fields for plain and

synchronized lyrics, but track availability and rights must still be evaluated. [8]

11.2 Search implementation

- Debounce user input to avoid issuing one request per keystroke.
- Cancel or ignore stale requests when a newer query supersedes an older one.
- Use provider pagination and merge/deduplicate results with provider-qualified keys.
- Expose source filter, genre/mood filters and a clear search reset action.
- Handle loading, empty, partial provider failure, rate limiting and offline states separately.
- Keep track cards accessible and show whether play launches full stream, preview, official player or external handoff.

11.3 Recommendations

Begin with transparent rule-based recommendations from listening history, saved tracks and selected genres. Add a recommendation backend only when there is enough legitimate interaction data. Include ways to dismiss recommendations and clear history. Do not infer sensitive traits from listening patterns or log unnecessary personal data.

12. Offline Support and Synchronization

Offline behavior requires explicit local data design when Supabase is the primary backend. Firestore can automatically cache documents and synchronize changes on supported mobile clients, while the same experience with Supabase should use a local store and synchronization service. [3]

12.1 Offline flow

- Read permitted cached metadata and user-owned cached records for immediate rendering.
- When offline, apply supported user actions locally and record them in a durable pending-operation queue.
- Give every operation an idempotency key and track pending, succeeded and failed states.

- On reconnection, confirm the user session, replay eligible operations, handle authorization failures and update the local cache.
- Use explicit conflict rules for playlist reorder/edit operations that could otherwise overwrite another device's changes.
- Retry transient failures with backoff; do not retry invalid data or authorization failures indefinitely.
- Clear or partition account-specific cache on sign-out, account switch or account deletion.

12.2 Offline boundaries

Cached playlist metadata does not mean its media is playable offline. Offline audio requires a separate, explicitly permitted download/storage strategy. Provider stream URLs may expire, and protected embedded players may require a connection. The UI must state these limitations honestly.

12.3 Realtime updates

Use Supabase Realtime for targeted user-data changes such as playlist or preference updates across devices where useful. Configure the required publication and confirm RLS access. Avoid subscribing to the entire public catalogue: paginate catalogue requests and cache permitted metadata instead.

13. Theme, Animation and Responsive Design

13.1 Theme modes and palettes

Theme	Direction	Implementation requirement
System	Follow the operating-system preference.	Update without a restart when supported.
Light	Pearl surfaces, dark legible text, restrained violet/cyan accent.	Check borders, text fields and disabled states.
Dark	Midnight surfaces, high contrast, controlled glow.	Avoid low-contrast gray text on dark panels.
Obsidian Aurora	Near-black with cyan and violet accents.	Use glow sparingly; selected controls need clear focus.
Pearl Prism	Cool pale surfaces and lilac borders.	Preserve readable body copy and icons.
Emerald Sound	Forest/teal and mint.	Keep success color distinct from selected state.
Velvet Sunset	Plum, rose and warm accent.	Maintain contrast on gradients and artwork.

13.2 Motion system

Interaction	Suggested behavior	Guardrail
Page transition	Fade plus modest slide, around 220–320 ms.	Respect reduce-motion preferences.
Card arrival	Subtle staggered fade/translation.	Do not delay interaction until all items animate.
Button tap	Ripple and small scale feedback.	Provide accessible focus/pressed states.
Dialog/bottom sheet	Short fade/scale or slide.	Maintain keyboard/focus and safe-area behavior.
Theme change	Animate colors and surfaces.	Avoid flashing or unreadable intermediate colors.
Artwork	Hero-style change and mild ambient glow.	Profile blur/shadow cost; avoid excessive repainting.
Lyrics	Highlight active timestamped line.	Update only the necessary subtree.
Loading	Skeleton for content; inline progress for small actions.	Never use a permanent loader for a failed request.

13.3 Responsive layout

Use SafeArea, LayoutBuilder, MediaQuery, Flexible, Expanded, Wrap, slivers and constrained content widths. Test narrow and larger phones, text scaling, landscape and supported tablet/desktop layouts. Ensure the keyboard does not hide form controls, long labels can wrap, grids adapt to available width and the mini-player does not collide with bottom navigation. Use semantic labels and a practical minimum touch target.

14. Privacy, Security and Reliability

14.1 Security checklist

- Use Supabase Auth identity as the basis for ownership checks.
- Enable RLS on every exposed table and validate allowed operations.
- Keep service-role/secret keys and provider-private credentials server-side.
- Validate user inputs on both client and server/database boundaries as appropriate.
- Limit Edge Function access and apply appropriate rate limiting to exposed endpoints.

- Use least privilege for database roles and storage policies.
- Remove credentials, tokens and sensitive provider responses from logs.
- Test cross-account isolation with at least two test users.
- Use development and production projects separately where practical.
- Create backup/recovery and incident handling procedures before production launch.

14.2 Privacy principles

Collect only data needed to operate the app. Listening history should have a clear retention and deletion behavior. Avoid logging sensitive personal data or inferring sensitive attributes from music choices. Provide a route to sign out and, if supported, delete an account. Document provider data usage and any third-party data transfers.

14.3 Resilience

Centralize error mapping for authentication, database and provider calls. Distinguish offline, timeout, permission denied, invalid response, empty catalogue and unavailable media. Retry transient requests with bounded backoff. Ensure subscriptions and animation/audio controllers are disposed correctly and that asynchronous callbacks cannot update dead widget state.

15. Testing Strategy and Acceptance Criteria

Test area	Required checks	Acceptance result
Static analysis	Format code and run Flutter analysis.	No unresolved analyzer errors; justified warnings documented.
Authentication	Register, verify, sign in, reset password, restore session, sign out.	Real Supabase operations pass and failure states are usable.
RLS	Anonymous access, owner access, cross-user denial, playlist-track ownership.	Unauthorized reads/writes are denied.
Database	Migrations, constraints, duplicate favorites, playlist reorder and deletion.	Schema is repeatable and integrity constraints behave as intended.
Music providers	Search, pagination, malformed data, rate limit, missing artwork, duplicate results.	Correct parsing, dedupe and error states.
Playback	Valid playable resource, pause/resume, seek, queue, repeat, shuffle, interruption.	Controls follow actual engine events.
Lyrics	Matched, missing, unsynced, synchronized and provider-error responses.	No fabricated lyrics; timing is correct when available.
Offline	Open cache, queue favorite, reconnect, duplicate retry, conflict and account switch.	No data loss or cross-account leakage in tested scenarios.
UI themes	Every main screen in all supported themes.	Legible contrast, no clipping and correct preference persistence.

Test area	Required checks	Acceptance result
Responsive UI	Small/narrow device, keyboard, orientation and large text.	No known overflow or inaccessible critical action.
Performance	Profile page scroll, artwork transition, splash and player updates.	No persistent jank; costly rendering is addressed.
Release build	Build supported targets with production configuration.	Build succeeds and configuration/secrets are reviewed.

15.1 Definition of done

A feature is complete only when it is connected to the intended real service, has loading/success/error/empty states, respects access control, works across the applicable themes and screen sizes, and has a relevant test. A visually correct button with no working action is not complete. A fake provider response used for a unit test is acceptable only when it is a test double and cannot silently replace a live provider in production.

16. Implementation Roadmap

Phase	Work packages	Exit condition
0. Audit	Inspect repository, dependencies, errors, data flows and prior implementation.	Written baseline and feature/file map.
1. Foundations	App bootstrap, routing, theme tokens, motion system and common widgets.	App compiles and navigation shell is stable.
2. Supabase	Project setup, migrations, constraints, RLS and security tests.	Schema deploys cleanly; user isolation tests pass.
3. Auth	Registration, verification, sign-in, reset, session and profile.	All account lifecycle tests pass against Supabase.
4. Catalogue	Provider adapters, normalized track model, pagination and attribution.	Live provider responses and capabilities verified.
5. Playback	Audio service, queue, real progress, errors and background support.	Valid media can be played and controlled on target device.
6. Lyrics and library	Lyrics adapter, favorites, playlists, history and theme preferences.	User changes persist and inaccessible content is handled.
7. Offline	SQLite cache, pending-operation queue and conflict policy.	Offline actions reconcile correctly after reconnect.
8. Finish	Responsive design, animations, accessibility and performance checks.	Main journeys pass regression and visual verification.
9. Release	Production configuration, build, security review and documentation.	Release checklist signed off with limitations recorded.

16.1 Recommended development order

Implement authentication and secure data foundations before polishing every card animation. Next, prove the provider adapter and playback capability on real media. Then connect favorites/playlists, implement local caching, and finish the visual system. This sequence reduces the risk of creating an impressive interface that is not backed by functioning services.

17. Risks, Constraints and Mitigations

Risk	Impact	Mitigation
Music API restrictions or changes	Some tracks may be preview-only, unavailable or rate-limited.	Build adapter isolation, validate current documentation and render capability-specific actions.
Copyright and lyrics permissions	Content may be technically accessible but not licensed for intended use.	Use permitted sources, attribution, external handoff and legal review before public launch.
False track count	Displayed count may include duplicates or stale cache.	Measure unique provider-keyed records and expose actual API/cache state in diagnostics.
Database access misconfiguration	Users could read or alter other users' data.	RLS by default, least-privilege grants and automated allow/deny tests.
Offline conflict	One device may overwrite another device's playlist changes.	Idempotent operations, version/conflict policy and a defined merge strategy.
Performance cost of glossy effects	Dropped frames, battery use or overheating.	Profile on physical devices and limit expensive blur, shadow and repeated repaint.
Provider secrets leaked to client	Unexpected quota use or unauthorized provider requests.	Keep secrets in Edge Functions/managed secret storage.
Unverified backend status	UI may look complete even though operations are not wired.	Separate screenshot evidence from tested behavior; require end-to-end acceptance tests.

18. Future Enhancements

- Smarter recommendations based on non-sensitive, user-controlled listening preferences.
- Expanded accessibility and localization, including English and Urdu interface strings where required.
- Optional provider-based sign-in and multi-factor authentication after the primary flow is stable.
- Carefully implemented audio visualizations based on data available from the active player.
- Playlist sharing with explicit visibility and access policies.

- Privacy-conscious crash reporting, playback-start success metrics and provider latency monitoring.
- Optional download support only for providers and tracks whose permissions allow it.
- Discovery collections that refresh from actual source APIs and support user dismissal or personalization reset.

Conclusion

LumaWave's supplied screenshots establish a coherent premium visual direction: a luminous introductory screen, glossy sign-in form, music-oriented home dashboard, search and discovery cards, personal library, playlist screens and theme preferences. The project's next step is to connect these visible interfaces to a secure and testable implementation.

Supabase is the recommended primary backend because PostgreSQL and Row Level Security suit the application's relational user data. A robust release still requires carefully configured authentication, tested migrations and policies, real provider integrations, truthful playback capability handling, rights-aware lyrics access, explicit offline synchronization and performance validation.

The final product should be judged by genuine behavior as well as appearance: user changes persist, unauthorized access is blocked, provider results are real, playback controls reflect the audio engine, offline status is accurate and every screen remains accessible and responsive. The attached screenshots form the visual appendix; implementation and backend claims should be marked complete only after the relevant tests are executed.

References and Technical Resources

[1] Supabase. "Use Supabase with Flutter" — official Flutter quickstart. https://supabase.com/docs/guides/getting-started/quickstarts/flutter

[2] Supabase. "Row Level Security" — database access policies. https://supabase.com/docs/guides/database/postgres/row-level-security

[3] Firebase. "Access data offline" — Firestore offline persistence and synchronization (used for comparison). https://firebase.google.com/docs/firestore/manage-data/enable-offline

[4] Supabase. "API Keys" — publishable versus secret/service-role key handling. https://supabase.com/docs/guides/getting-started/api-keys

[5] Audius Developer Docs. "API Reference" — catalogue and stream API surface. https://docs.audius.co/api/

[6] Apple Developer. “iTunes Search API: Overview” — catalogue API and promotional content conditions. https://developer.apple.com/library/archive/documentation/AudioVideo/Conceptual/iTuneSearchAPI/

[7] Flutter. “Performance best practices” — rendering and animation guidance. https://docs.flutter.dev/perf/best-practices

[8] LRCLIB. “API Documentation” — lyrics API data structure. https://lrclib.net/docs

[9] Supabase. “Securing your data” — grants, RLS and privileged key considerations. https://supabase.com/docs/guides/database/secure-data

[10] Flutter. “App architecture” — separation of UI and data concerns. https://docs.flutter.dev/app-architecture

[11] Supabase. “Edge Functions” — server-side TypeScript functions and integrations. https://supabase.com/docs/guides/functions

Appendix A. Supplied UI Screenshots

The following appendix contains all 12 images from the uploaded screenshots.zip archive. Images are embedded from the supplied 720 × 1600 pixel JPEGs without cropping; they are scaled proportionally for page fit. The accompanying captions describe visible UI content and do not certify backend functionality.

PROJECT SCREENSHOT 01 OF 12

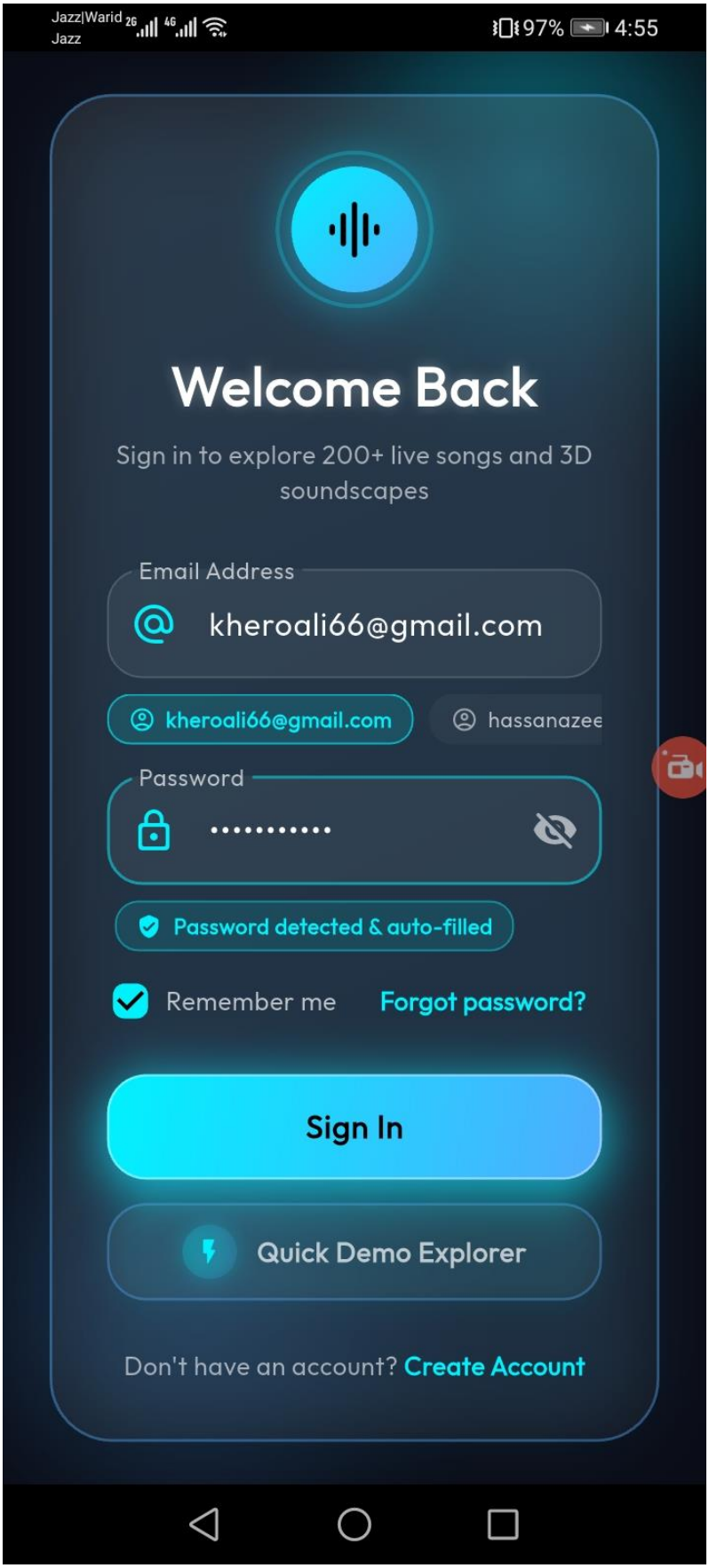


Figure A1. Sign-in screen showing email and password fields, Remember me, password recovery and Create Account navigation.

Source: user-supplied screenshot 1 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 02 OF 12

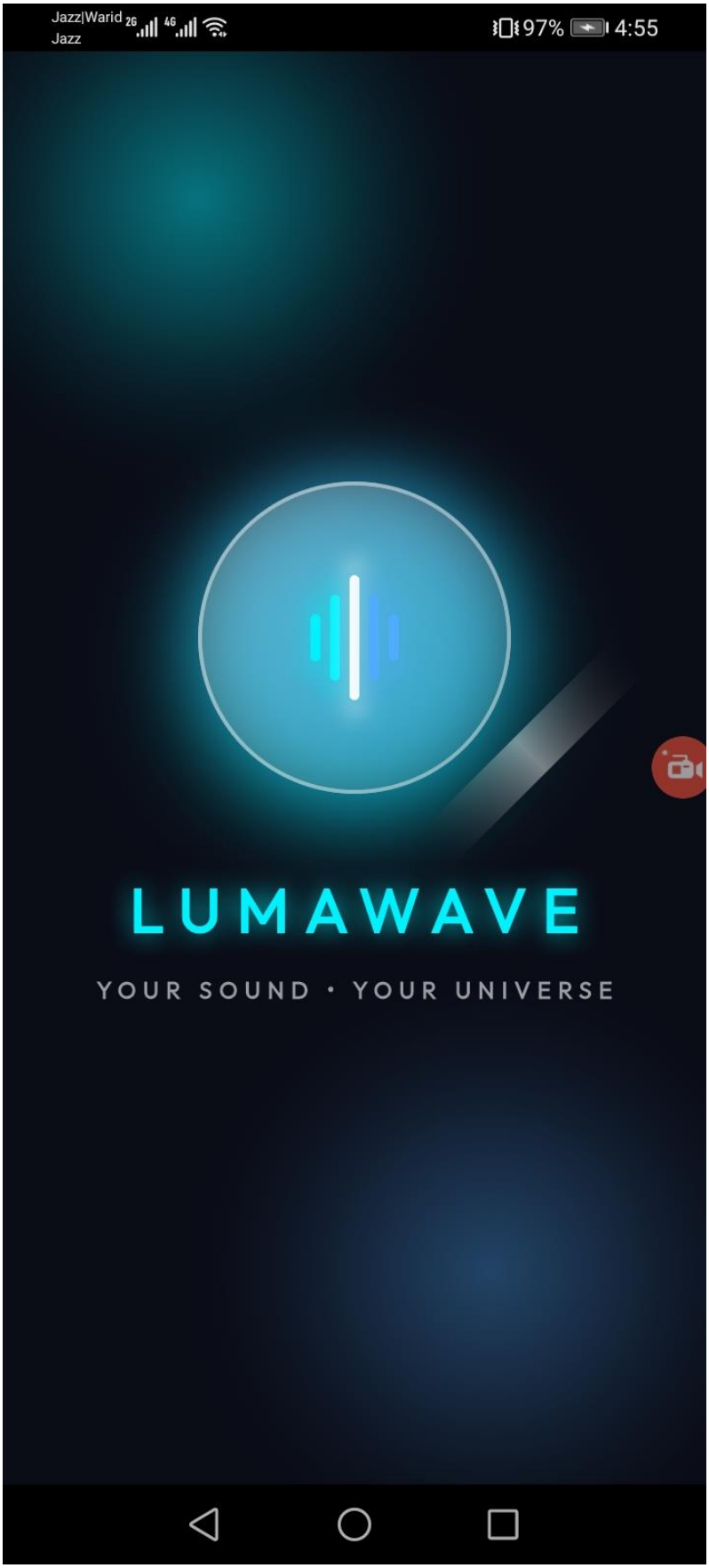


Figure A2. LumaWave splash screen with luminous waveform emblem, wordmark and tagline.

Source: user-supplied screenshot 2 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 03 OF 12

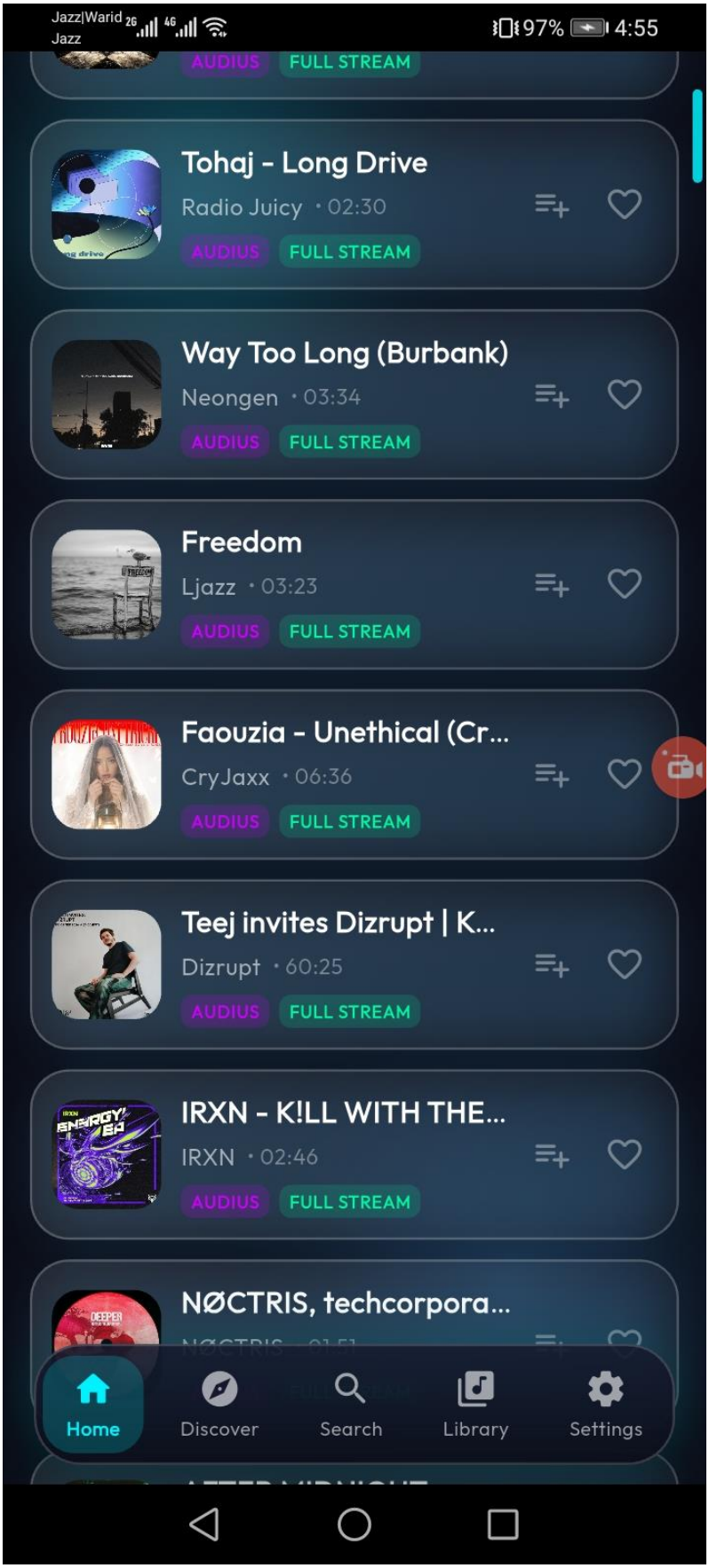


Figure A3. Track list / catalogue view with artwork cards, provider labels and playback-capability badges.

Source: user-supplied screenshot 3 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 04 OF 12

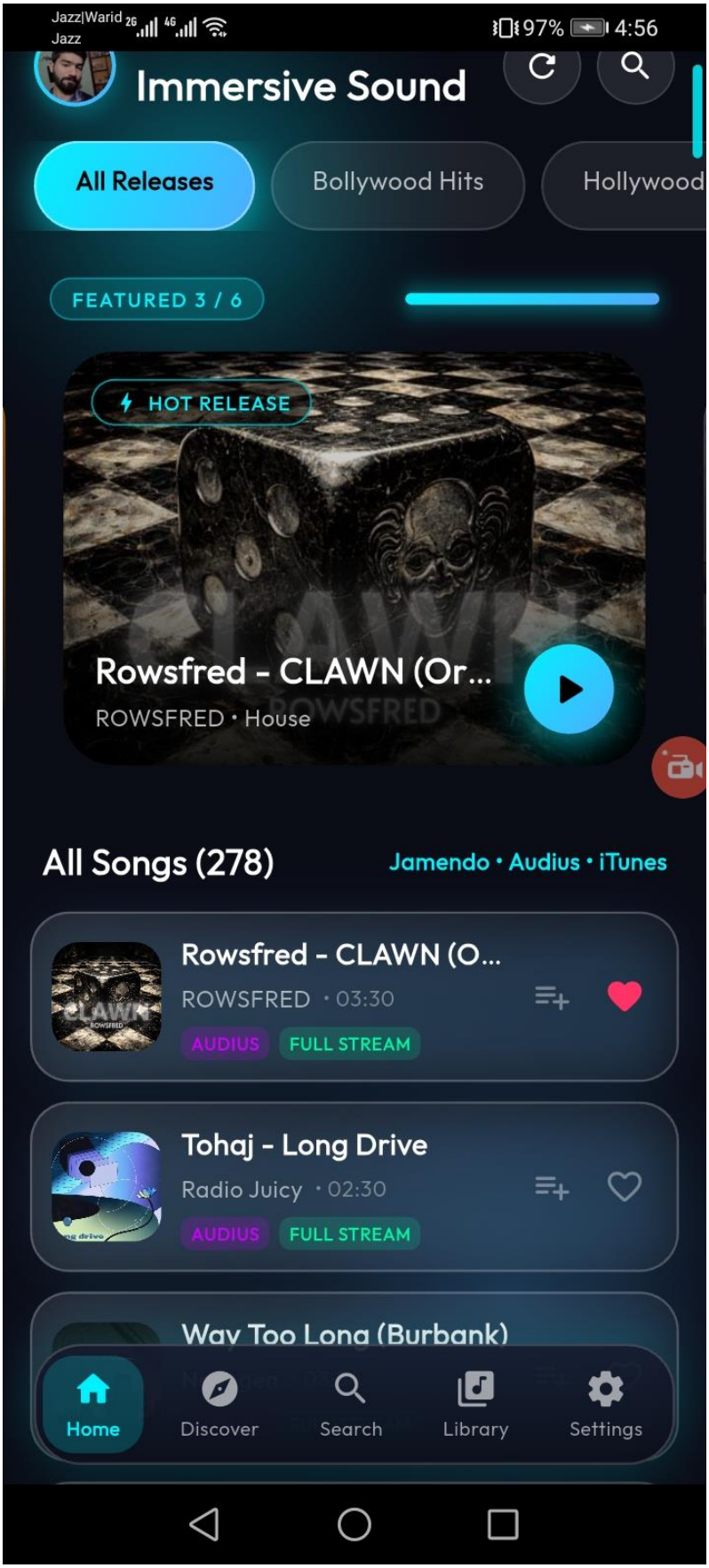


Figure A4. Immersive Sound catalogue view with release filters, featured artwork and track results.

Source: user-supplied screenshot 4 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 05 OF 12

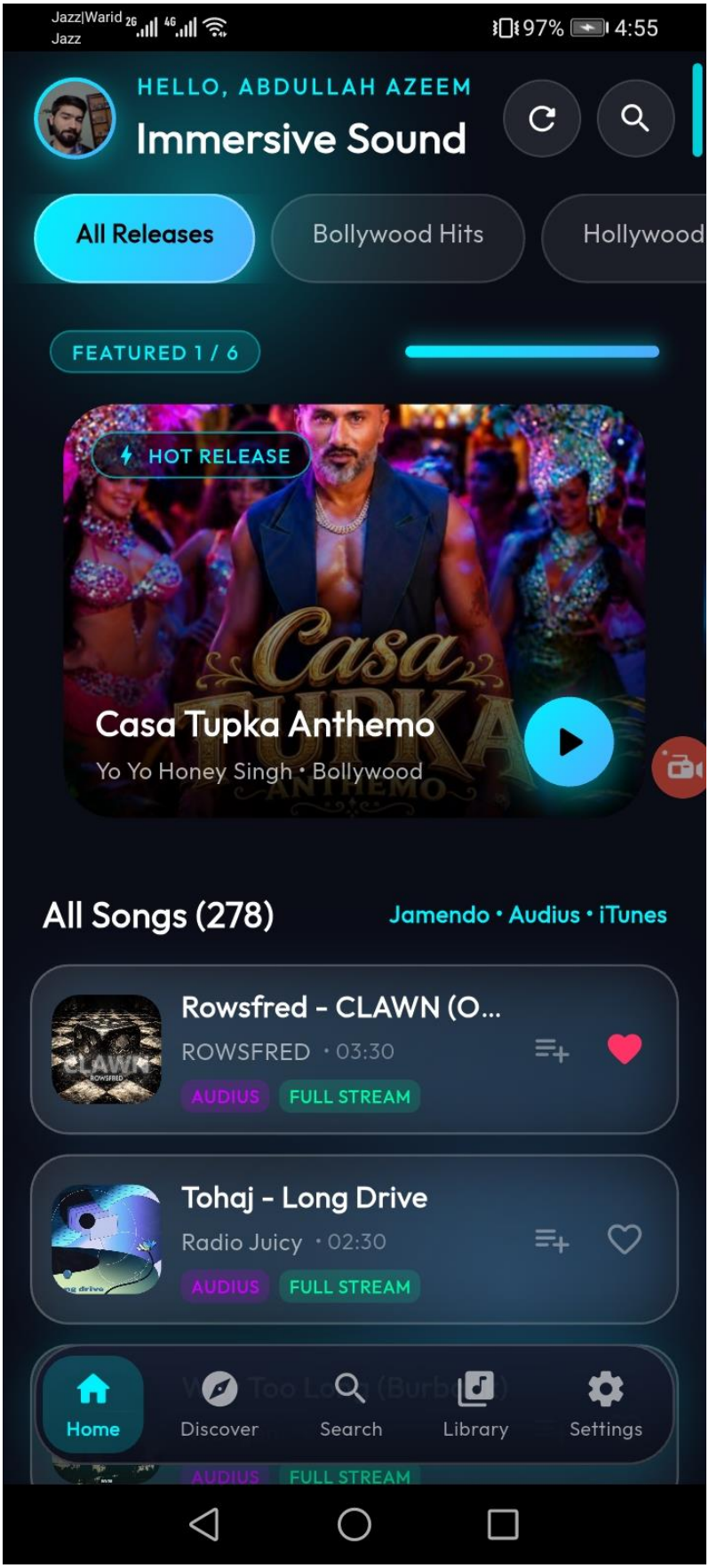


Figure A5. Home dashboard with personalized greeting, featured music and track rows.

Source: user-supplied screenshot 5 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 06 OF 12

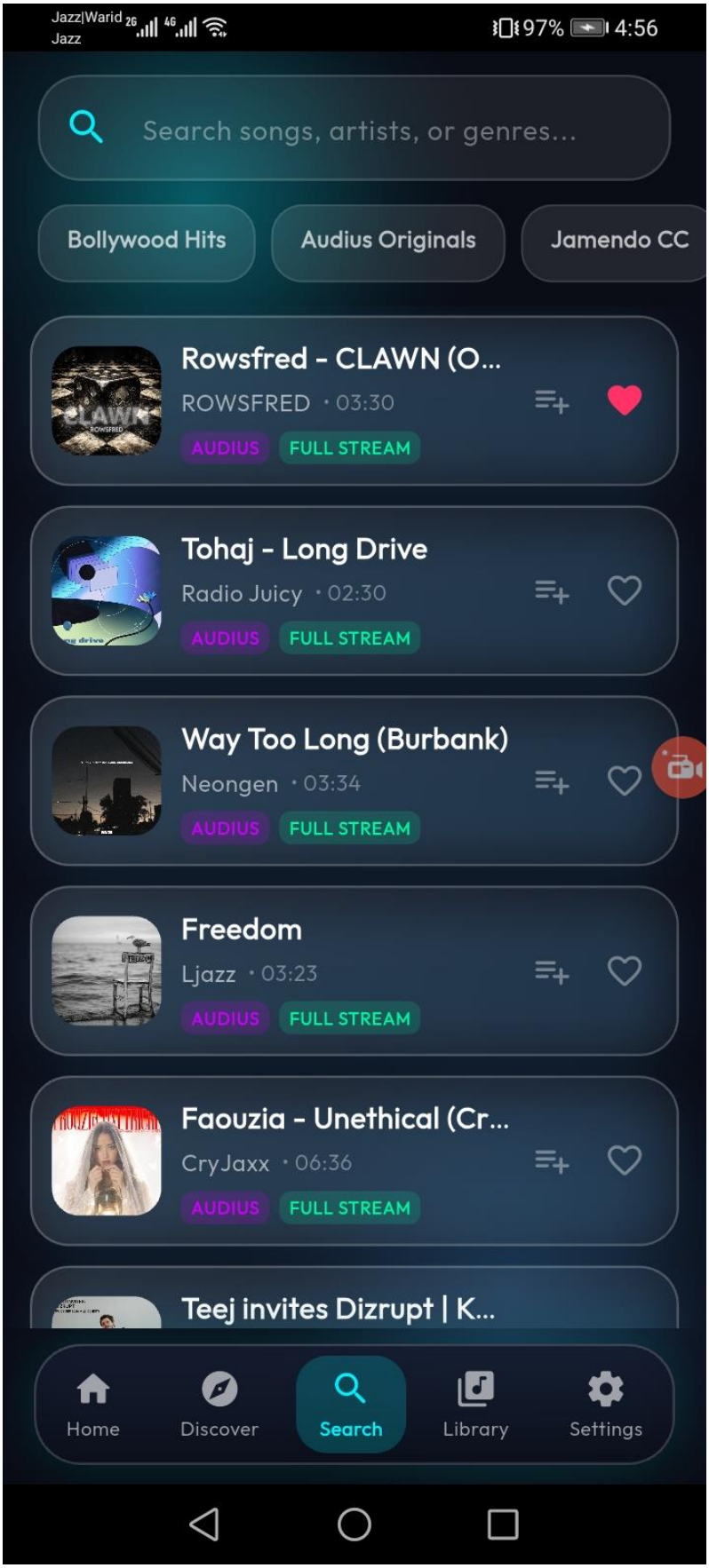


Figure A6. Search screen with search input, category filters and results list.

Source: user-supplied screenshot 6 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 07 OF 12

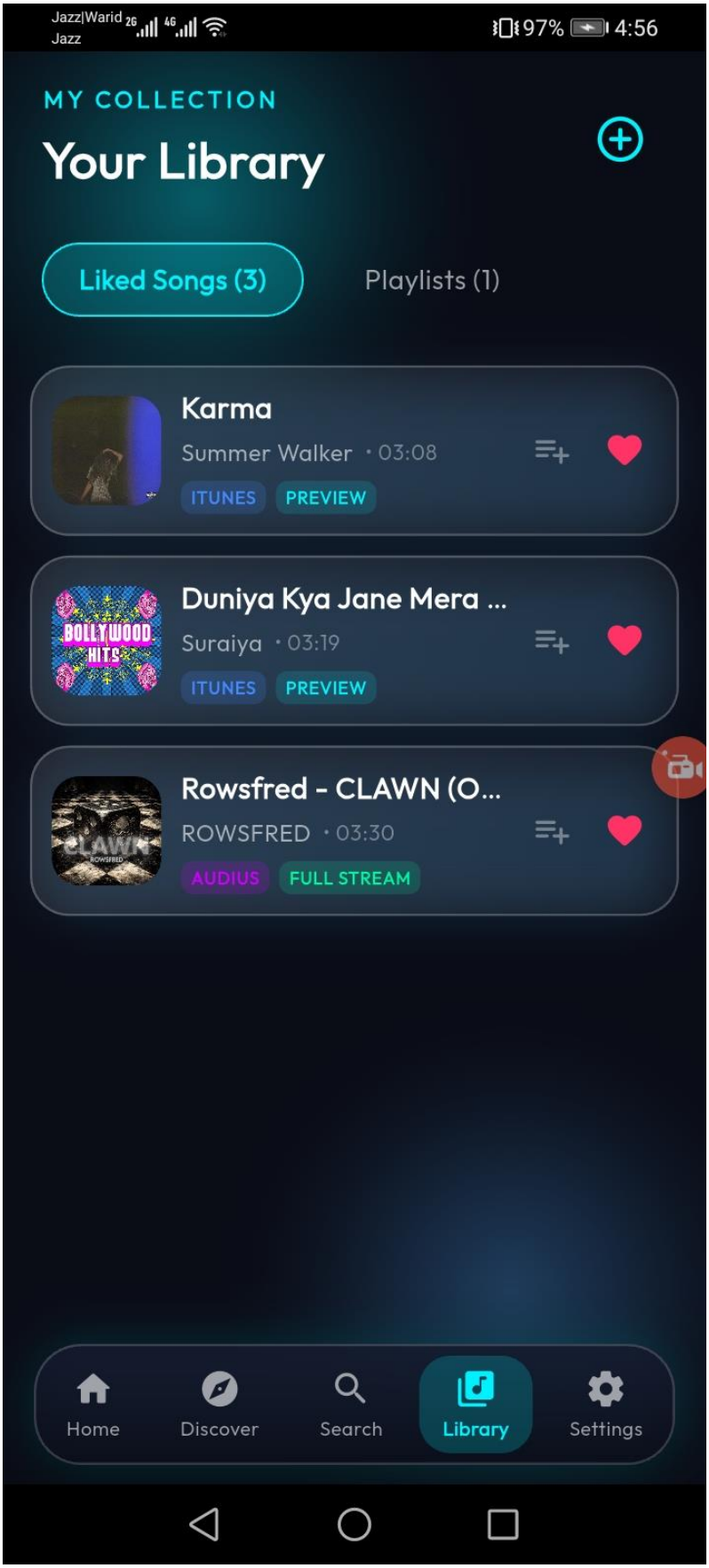


Figure A7. Your Library view with liked songs and collection tabs.

Source: user-supplied screenshot 7 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 08 OF 12

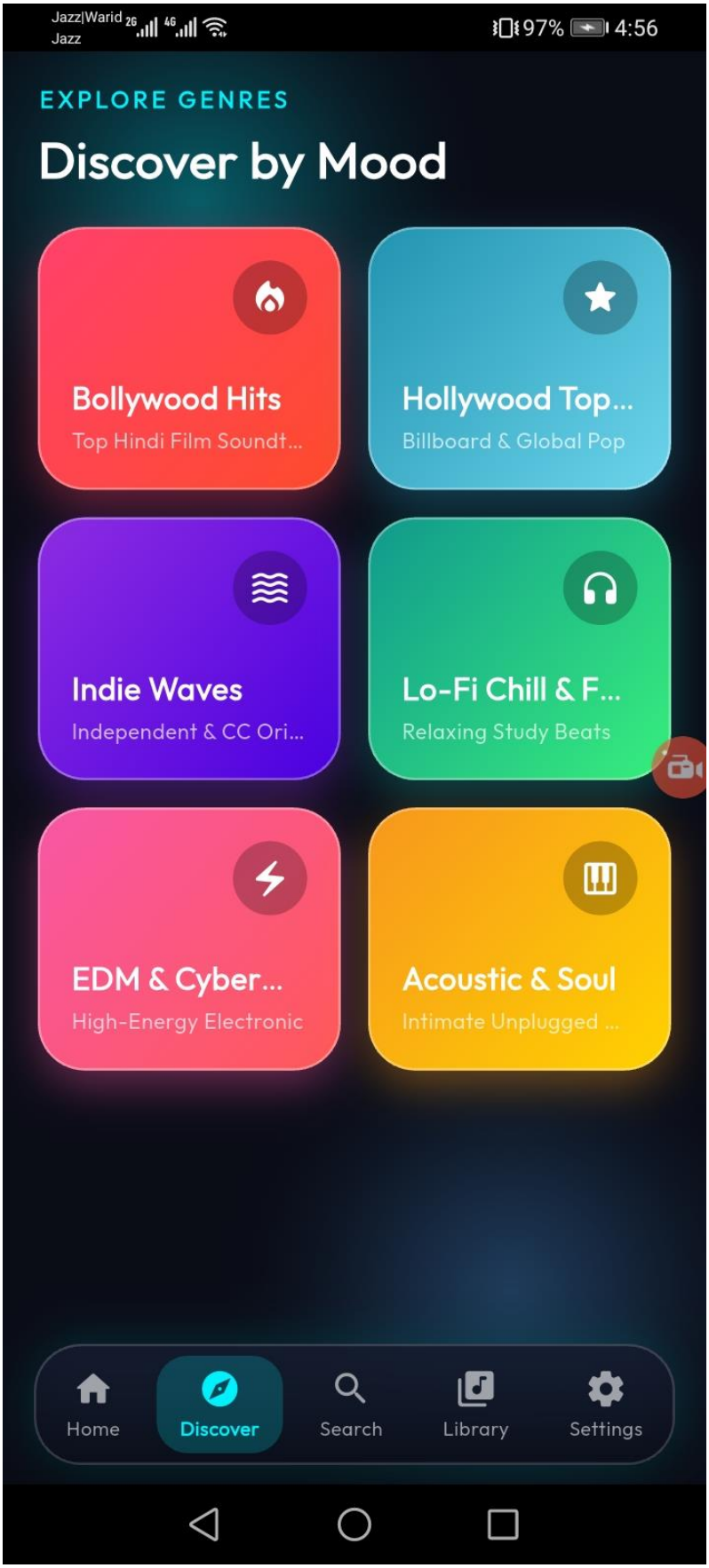


Figure A8. Discover by Mood screen with genre and mood category cards.

Source: user-supplied screenshot 8 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 09 OF 12

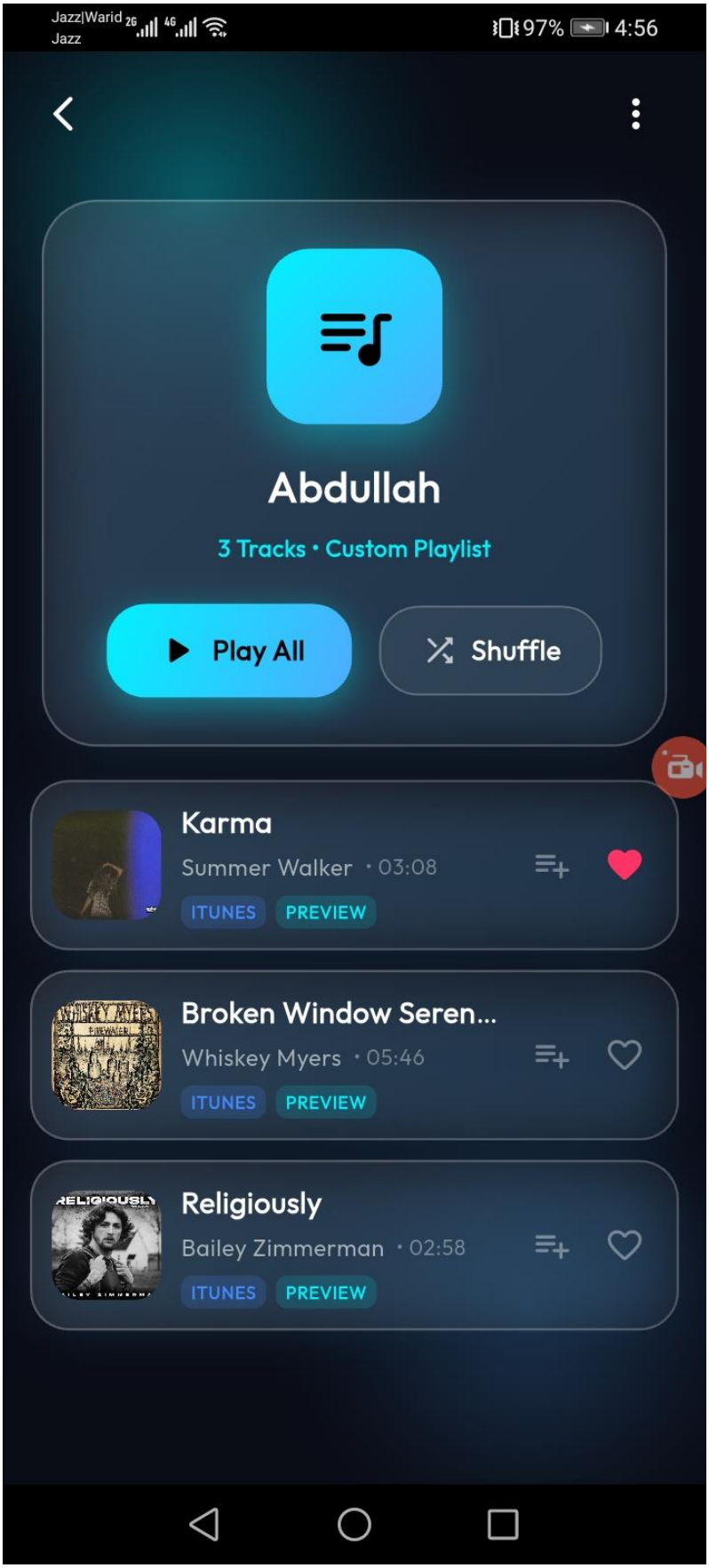


Figure A9. Playlist details screen with playlist artwork, track count, Play All and Shuffle controls.

Source: user-supplied screenshot 9 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 10 OF 12

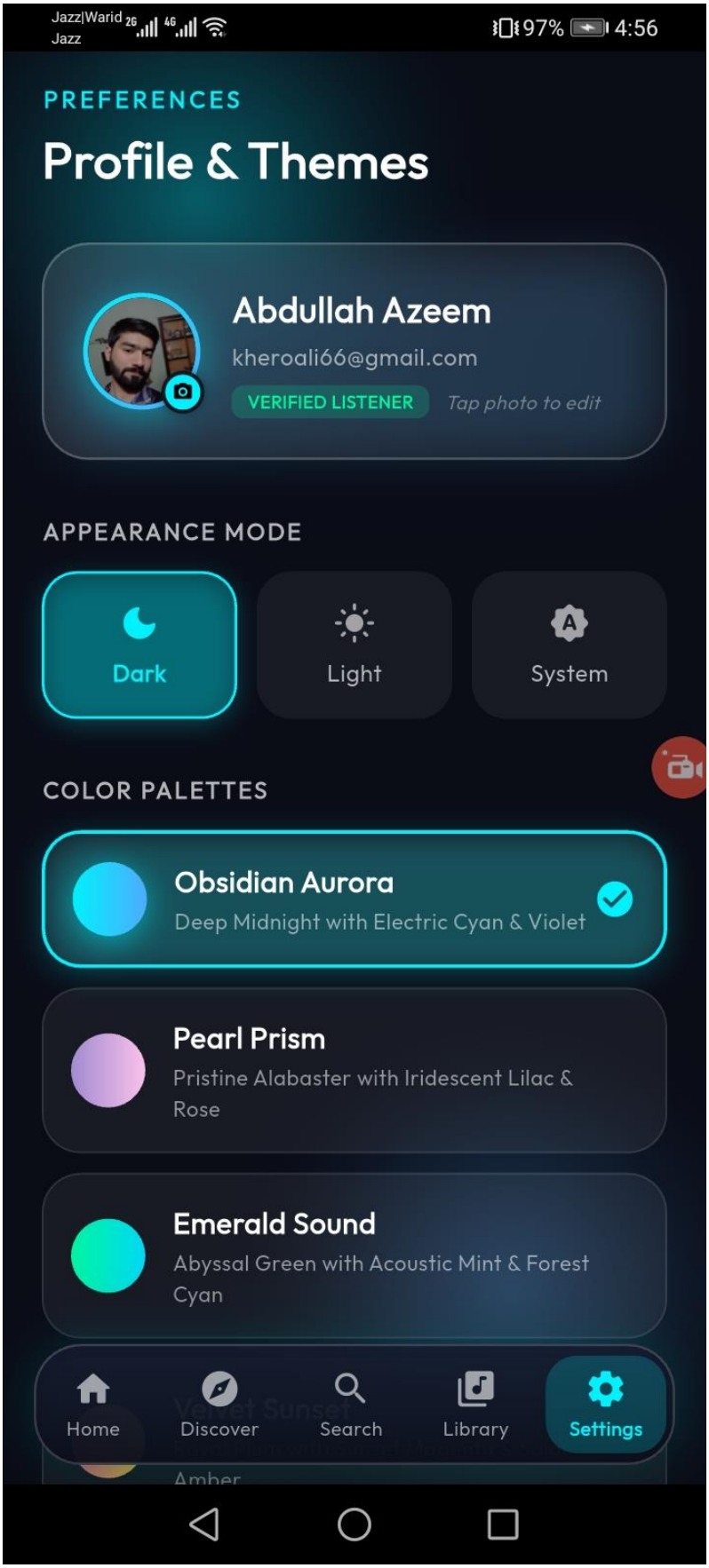


Figure A10. Profile & Themes screen with profile card and appearance-mode choices.

Source: user-supplied screenshot 10 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 11 OF 12

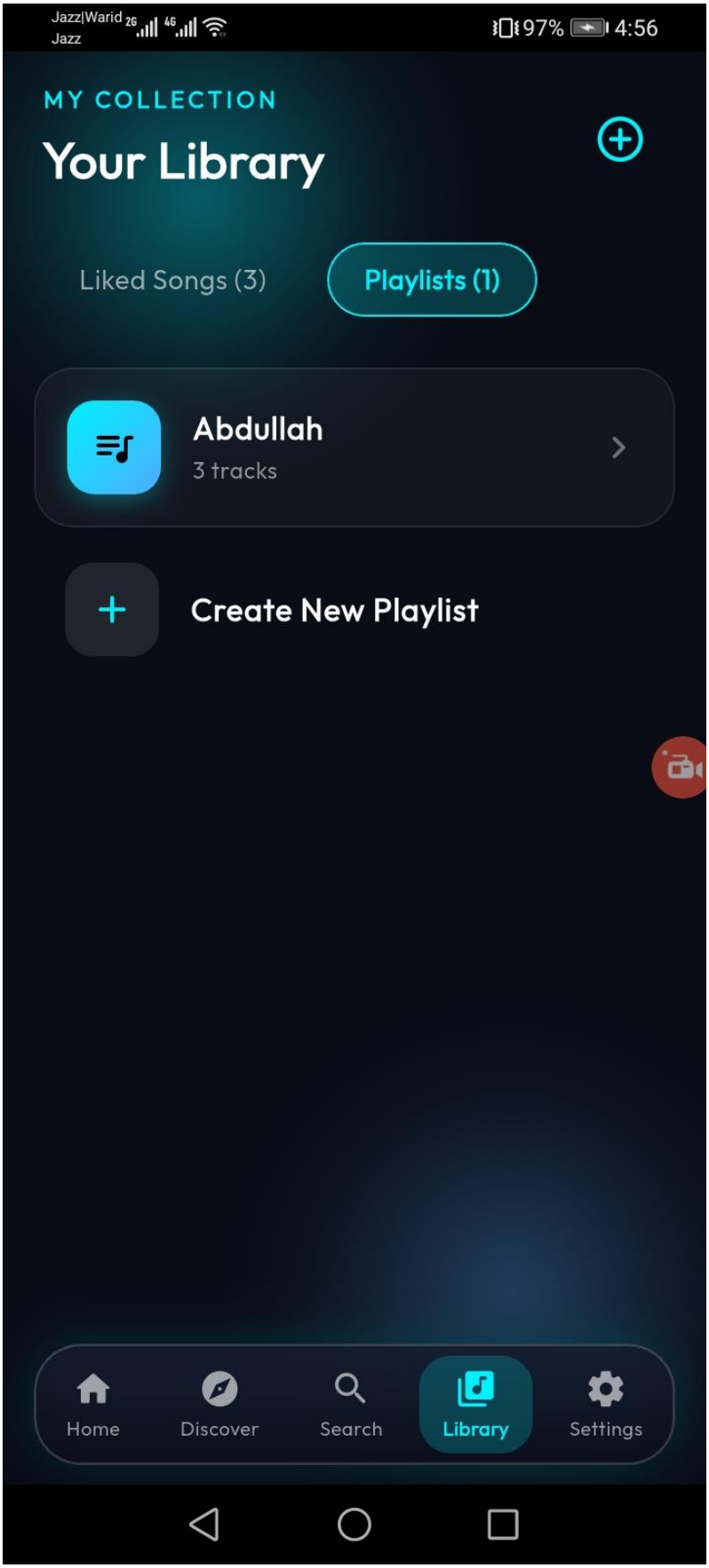


Figure A11. Library playlist collection view with an existing playlist and Create New Playlist action.

Source: user-supplied screenshot 11 of 12 • Original resolution: 720 × 1600 px

PROJECT SCREENSHOT 12 OF 12

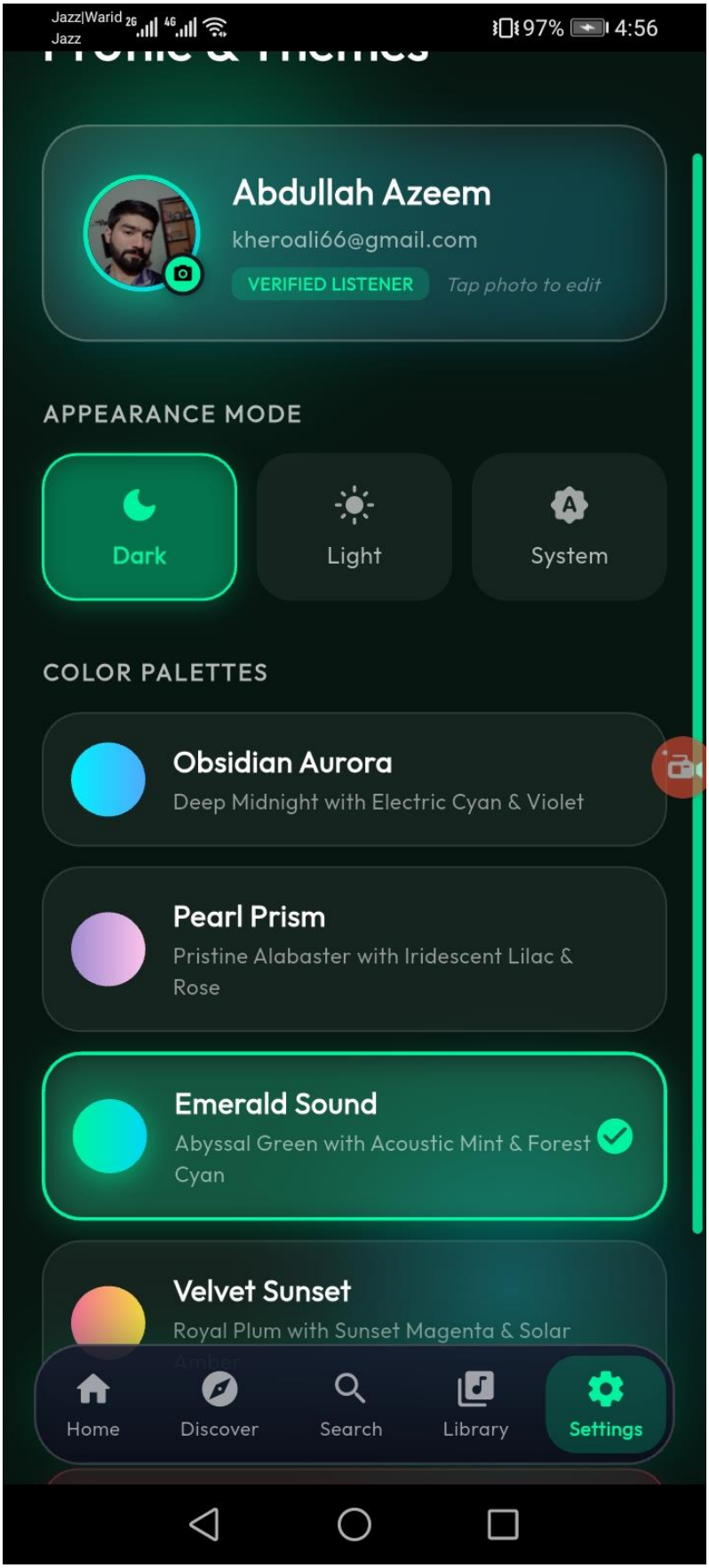


Figure A12. Profile & Themes screen showing multiple selectable colour palettes.

Source: user-supplied screenshot 12 of 12 • Original resolution: 720 × 1600 px